

Software Testing Assessment (M3-A1)

Testing on Live Application & Full Life Cycle Deliverables

Module: M3 - Testing on Live Application | Domain: Food Delivery Mobile & Web Platform | Status: Complete Bundled Test Pack

SECTION A — CONCEPT APPLICATION

Scenario 1 (S1): Agile SDLC Testing & Sprint Criteria

Scenario: Mobile food delivery QA team in an Agile (Scrum) 2-week sprint model. PM needs to know when testing starts, entry/exit criteria, and differences from Waterfall.

In the Agile (Scrum) model, testing begins at the very start of the sprint during sprint planning and requirement refinement (Sprint Day 1).

- **Entry Criteria:** (1) User stories must meet the Definition of Ready (DoR) with clear acceptance criteria; (2) The test environment must be configured and deployed with a testable build.
- **Exit Criteria:** (1) 100% of planned test cases executed with 0 open High/Critical severity defects; (2) All acceptance criteria for user stories are met and verified (Definition of Done).

Agile vs. Waterfall Difference: Waterfall isolates testing into a distinct stage after full development, making defect discovery late and expensive. Agile integrates continuous testing alongside development within two-week iterations, ensuring immediate feedback and early bug prevention.

Scenario 2 (S2): Non-Functional Testing & Testing Principles

Scenario: Reviewing test plan for payment gateway expecting 5,000 concurrent transactions during peak hours. Junior tester skipped non-functional tests.

Three Critical Non-Functional Testing Types:

- **Performance/Load Testing:** Validates if the payment system handles the 5,000 concurrent transaction load without crashing or response degradation.
- **Security Testing:** Ensures sensitive cardholder and transaction data (PCI-DSS compliance) are encrypted and secure from breach during transit.
- **Reliability/Stress Testing:** Verifies system stability under peak load stress and tests auto-recovery from payment gateway timeouts.

Testing Principle Applied: *Principle 2: Exhaustive testing is impossible.* Testing every combination of payment method, currency, network speed, user state, and bank API payload is mathematically infinite. QA must use risk-based testing and equivalence technique profiling to focus on critical paths.

Scenario 3 (S3): EP & BVA for Promo Code Field

Scenario: Designing test cases for 'Apply Promo Code' field accepting 6–10 alphanumeric characters.

- **Equivalence Partitions (Length & Type):**
 - *Valid Partitions:* Alphanumeric strings of length 6 to 10 characters (e.g., SAVE10, F00D2026).
 - *Invalid Partitions:* Length < 6 characters; Length > 10 characters; Contains special characters/spaces; Numeric/Special-only strings if restricted.

- **Exact Boundary Values (Length):** 5 (Invalid), 6 (Valid Lower Boundary), 7 (Valid Interior), 9 (Valid Interior), 10 (Valid Upper Boundary), 11 (Invalid).
- **Better Technique for Defect Coverage: Boundary Value Analysis (BVA)** offers higher defect coverage for length validation because off-by-one coding errors occur most frequently at the minimum and maximum threshold limits (length == 6 vs length > 6).

Scenario 4 (S4): Defect Classification & Prioritization

Scenario: One day before release: (A) App crashes on 'Place Order' on Android affecting 30% of users; (B) Restaurant logo slightly misaligned on iPhone SE.

- **Defect Classification:**
 - *Defect A (Android App Crash on Order):* **Severity: Critical/High | Priority: High/P1.**
 - *Defect B (Restaurant Logo Misaligned):* **Severity: Low | Priority: Low/P4.**
- **Severity vs. Priority:** *Severity* measures the technical impact of a defect on application functionality (app crash = high structural damage). *Priority* measures business urgency for fixing the issue (crashing order checkout directly blocks revenue).
- **Which to Fix First: Defect A must be fixed first.** It completely blocks core revenue generation for 30% of users, whereas Defect B is a minor cosmetic defect that does not block ordering.

Scenario 5 (S5): Cross-Browser & Responsive Validation

Scenario: 'Track Order' map widget works on Chrome but missing on Firefox. Checkout layout collapses on 768px tablet.

- **Testing Types Identified:**
 1. *Cross-Browser Testing:* Validates functional and rendering consistency across different rendering engines (Chrome/Blink vs. Firefox/Gecko).
 2. *Responsive Web Design (RWD) / Cross-Device Testing:* Validates UI layout, CSS grid behavior, and usability across dynamic viewport breakpoints (768px tablet layout).
- **Specific Checks:**
 - *Cross-Browser:* JS console error logs, third-party map API script execution, DOM element visibility on Firefox.
 - *Responsive:* Flexbox/Grid CSS breaking points, touch target tap sizes, overlay overlaps at 768px.
- **Tools/Methods:** BrowserStack or Sauce Labs for multi-browser grid emulation; Chrome DevTools Responsive Mode for viewport testing; Jira for bug reporting with console logs.

Scenario 6 (S6): Backend SQL Validation & Data Integrity

Scenario: Validating backend database after 5 test orders because confirmation email shows wrong address.

SQL Query Structure:

```
-- Query to compare placed order delivery address with user profile address
SELECT
    o.order_id,
    o.user_id,
    o.delivery_address,
    u.registered_address
FROM orders o
JOIN users u ON o.user_id = u.user_id
WHERE o.order_id = 10051;
```

Data Verification vs. Data Validation:

- **Data Validation:** Checks whether data meets predefined formatting rules and constraints *before or during insertion* (e.g., verifying address field is non-null and under 250 characters).
- **Data Verification:** Checks whether the stored backend data *accurately reflects* the exact user input submitted on the UI front-end (e.g., executing the SQL query above to verify the stored database string matches the UI input field).

SECTION B — PRACTICAL TESTING TASKS

Task 1: Test Plan & Objectives — Login Module

1. Measurable Test Objectives (Aligned to STLC Stages)

- **Requirement Analysis:** Verify 100% of user stories related to authentication against business requirements and create an ambiguity log within 2 days.
- **Test Planning:** Complete test case design covering 100% functional requirements with a minimum test design pass rate of 95% during peer review.
- **Test Execution:** Execute 100% of planned test cases with zero unresolved Critical (S1) or High (S2) defects prior to sprint release sign-off.
- **Test Closure:** Deliver a comprehensive Test Summary Report documenting execution metrics, defect density, and residual risk within 24 hours of execution completion.

2. Test Scope

In-Scope: (1) Mobile number / Email and Password authentication flow; (2) Social Sign-In (Google and Apple OAuth integration); (3) "Forgot Password" OTP generation and password reset process.

Out-of-Scope: (1) Backend user database schema migration testing; (2) Payment gateway authorization after login; (3) Localization and translation into languages other than English.

3. Entry & Exit Criteria

Entry Criteria: (1) Deployment of the stable Login build to Staging Environment; (2) Acceptance criteria and test data prepared and verified.

Exit Criteria: (1) 100% planned test cases executed with a minimum 98% pass rate; (2) All S1 (Blocker) and S2 (Critical) defects resolved, retested, and closed.

4. Risks & Mitigations

Risk ID	Risk Description	Mitigation Strategy
R-01	Third-party SMS OTP gateway downtime delaying automated login testing.	Implement mock OTP service/stubs in QA environment for deterministic testing.
R-02	Unstable Staging Environment deployment causing false negative test failures.	Set up health-check scripts and freeze deployment builds during active execution.
R-03	Test account lockout due to rate-limiting rules during automated execution.	Maintain dynamic test user pool and implement DB reset scripts prior to test runs.

Task 2: EP & BVA Test Cases — Delivery Address Form

Pincode Field (6-digit numeric): Valid = 6 digits; Invalid = <6, >6, Alphabetic, Special. Boundaries = 5 (Inv), 6 (Val), 7 (Inv).

Phone Field (10-digit numeric): Valid = 10 digits; Invalid = <10, >10, Alphabetic, Special. Boundaries = 9 (Inv), 10 (Val), 11 (Inv).

TC ID	Description	Input Data	Expected Result	Status	Type
TC_ADDR_01	Valid 6-digit Pincode	Pincode: 380015	Address saved successfully.	PASS	EP / Pos
TC_ADDR_02	Invalid 5-digit Pincode	Pincode: 38001	Error: "Must be 6 digits".	PASS	BVA / Neg
TC_ADDR_03	Invalid 7-digit Pincode	Pincode: 3800159	Error: "Must be 6 digits".	PASS	BVA / Neg
TC_ADDR_04	Pincode with letters	Pincode: 3800AB	Non-numeric input blocked.	PASS	EP / Neg
TC_ADDR_05	Pincode with special chars	Pincode: 3800#1	Non-numeric input blocked.	PASS	EP / Neg

TC ID	Description	Input Data	Expected Result	Status	Type
TC_ADDR_06	Empty Pincode	Pincode: [Blank]	Error: "Pincode is required".	PASS	EP / Neg
TC_ADDR_07	Valid 10-digit Phone	Phone: 9876543210	Form saved successfully.	PASS	EP / Pos
TC_ADDR_08	Invalid 9-digit Phone	Phone: 987654321	Error: "Must be 10 digits".	PASS	BVA / Neg
TC_ADDR_09	Invalid 11-digit Phone	Phone: 98765432101	Error: "Max 10 digits".	PASS	BVA / Neg
TC_ADDR_10	Phone with letters	Phone: 987654321A	Non-numeric input blocked.	PASS	EP / Neg
TC_ADDR_11	Phone with spaces	Phone: 98765 43210	Error or auto-stripped spaces.	PASS	EP / Neg
TC_ADDR_12	Empty Phone Number	Phone: [Blank]	Error: "Phone is required".	PASS	EP / Neg

Task 3: Requirements Traceability Matrix — Order Placement Flow

Req ID	Requirement Description	Test Scenario IDs	Defect ID	Logged Defect Summary
FR-ORD-01	Add item to cart	TS_ORD_01_01, TS_ORD_01_02	—	None
FR-ORD-02	Apply coupon code	TS_ORD_02_01, TS_ORD_02_02	DEF-101	Discount calculation wrong for % coupons (S2 - Major).
FR-ORD-03	Select payment method	TS_ORD_03_01, TS_ORD_03_02	—	None
FR-ORD-04	Place order	TS_ORD_04_01, TS_ORD_04_02	DEF-102	App crashes on selecting Cash on Delivery option (S1 - Blocker).
FR-ORD-05	View confirmation screen	TS_ORD_05_01, TS_ORD_05_02	—	None
FR-ORD-06	Check order history	TS_ORD_06_01, TS_ORD_06_02	DEF-103	Order history time in UTC instead of local timezone (S4 - Minor).

Task 4: Web & Database Testing — Checkout Page

1. Five Web Application UI Checks

- **Form Validation:** Verify mandatory fields (Address, Payment) show inline error messages when submitted blank.
- **Navigation Links:** Verify "Back to Cart" and "Terms & Conditions" hyperlinks navigate correctly without broken 404 links.
- **Data Submission:** Verify clicking "Place Order" disables submit button to prevent double form submission.
- **UI Element Rendering:** Confirm cart summary, tax breakdown, tip selection, and total render without overlapping text.
- **Error Handling:** Verify user-friendly error banner appears when payment transaction fails due to card rejection.

2. Cross-Browser Scenarios

- **TS_CB_01 (Chrome):** Verify checkout layout alignment, CSS rendering, sticky footer, and dynamic cart calculations on Chrome (Blink).
- **TS_CB_02 (Firefox):** Verify payment modal triggers and cross-origin iframe security execution on Mozilla Firefox (Gecko).
- **TS_CB_03 (Edge):** Verify input field auto-complete capabilities, address form rendering, and gateway redirect on MS Edge.

3. SQL Validation Queries

```
-- Query 1: Validate Order Total stored in Database matches Checkout UI
SELECT order_id, user_id, total_amount, tax_amount, delivery_fee
FROM orders WHERE order_id = 50012;

-- Query 2: Validate Applied Coupon Code stored in Database
SELECT order_id, coupon_code, discount_amount
FROM order_discounts WHERE order_id = 50012;

-- Query 3: Validate Selected Payment Method stored in Database
SELECT order_id, payment_method, payment_status, transaction_id
FROM payments WHERE order_id = 50012;
```

4. Defect Report (UI-DB Mismatch)

DEFECT REPORT: BUG-ORD-309 — Discount Mismatch Between UI and Database

Defect ID: BUG-ORD-309

Module: Checkout / DB Sync

Severity: S2 (Major)

Priority: P1 (High)

Environment: Staging Web App / Chrome v126 / MySQL 8.0

Steps to Reproduce:

1. Add items worth \$50.00 to cart.
2. Navigate to Checkout page & apply promo code "FLAT10" (\$10.00 discount).
3. Verify UI displays Final Total as \$40.00 (Correct).
4. Click "Place Order" and complete transaction.
5. Run query: `SELECT total_amount, discount_amount FROM orders WHERE order_id = [NEW_ID];`

Expected Result: DB orders table should record `discount_amount = 10.00` and `total_amount = 40.00`.

Actual Result: DB orders table records `discount_amount = 0.00` and `total_amount = 50.00` (Full amount charged without discount).

SECTION C — MINI CAPSTONE PROJECT

End-to-End Manual Testing Project — Food Delivery Web Portal

1. One-Page Test Plan Summary

Objectives: Validate end-to-end user workflows across Registration, Login, Restaurant Discovery, Order Placement, and Tracking features, ensuring 100% functional accuracy, cross-browser compatibility, and backend DB data integrity.

Scope: *In-Scope:* Registration, Login, Search/Filter, Cart Management, Order Placement, Live Tracking, SQL Verification.

Out-of-Scope: Admin Portal, Driver App, Gateway Settlement API.

Entry/Exit: Entry requires stable build & smoke test pass. Exit requires 100% execution, 0 open S1/S2 defects, and complete RTM coverage.

Risks & Mitigations: Live location API failing (mitigated via mock responses); Wallet balance depletion (mitigated via DB seed scripts); DB response lag under load (mitigated via isolated QA database instance).

2. Test Case Suite (20 Cases including EP, BVA & Decision Table)

TC ID	Module	Description	Inputs / Action	Expected Result
TC_REG_01	Register	Valid User Registration	Valid Name, Email, Phone, Pass	Account created; redirect home.
TC_REG_02	Register	Invalid Email Format	Email: user.domain.com	Error: "Invalid email format".
TC_REG_03	Register	Password Min Length (8)	Password: Pass12 (6 chars)	Error: "Min 8 chars required".
TC_REG_04	Register	Password Valid Boundary	Password: Pass1234 (8 chars)	Accepted successfully.
TC_LGN_01	Login	Valid Login	Registered Email & Password	Login successful; token generated.
TC_LGN_02	Login	Unregistered Email	Email: unreg@food.com	Error: "Invalid credentials".
TC_LGN_03	Login	Incorrect Password	Valid Email + Wrong Password	Error: "Invalid credentials".
TC_BRW_01	Browse	Search Restaurant Name	Query: "Pizza Hut"	Matches displayed in list.
TC_BRW_02	Browse	Filter by Cuisine	Filter: "Italian"	Italian restaurants displayed.
TC_BRW_03	Browse	Non-existent Cuisine	Search: XYZ123	Display "No restaurants found".
TC_ORD_01	Order	Add Item to Cart	Click "Add" on Pizza item	Cart count & total update.
TC_ORD_02	Order	Max Quantity Boundary	Item Quantity = 10 (Max)	Quantity set to 10 successfully.
TC_ORD_03	Order	Exceed Max Quantity	Item Quantity = 11	Error: "Max 10 units per item".
TC_ORD_04	Dec Table	Coupon: Valid + Cart >= \$20	Coupon SAVE5, Cart = \$25	\$5 Discount applied.
TC_ORD_05	Dec Table	Coupon: Valid + Cart < \$20	Coupon SAVE5, Cart = \$15	Error: "Min order amount \$20".
TC_ORD_06	Dec Table	Coupon: Expired Code	Coupon EXPIRED, Cart = \$30	Error: "Coupon code expired".
TC_ORD_07	Dec Table	Payment: COD + Cart <= \$100	Method: COD, Cart = \$50	Order placed successfully.
TC_ORD_08	Dec Table	Payment: COD + Cart > \$100	Method: COD, Cart = \$150	Error: "COD not allowed > \$100".
TC_TRK_01	Tracking	View Live Status	Navigate to Track Order	Status shows "Preparing Food".

TC ID	Module	Description	Inputs / Action	Expected Result
TC_TRK_02	Tracking	Refresh Status Update	Order dispatched	Status updates: "Out for Delivery".

3. Requirements Traceability Matrix (RTM)

Req ID	Requirement Description	Test Case IDs	Defect ID
REQ-01	User Registration	TC_REG_01 - 04	DEF-CAP-01
REQ-02	User Authentication / Login	TC_LGN_01 - 03	—
REQ-03	Restaurant & Cuisine Search	TC_BRW_01 - 03	DEF-CAP-02
REQ-04	Cart & Item Management	TC_ORD_01 - 03	—
REQ-05	Coupon & Payment Rules	TC_ORD_04 - 08	DEF-CAP-03, DEF-CAP-04
REQ-06	Order Tracking & Status	TC_TRK_01 - 02	DEF-CAP-05

4. Defect Log (5 Capstone Defects)

Defect ID	Module	Steps & Actual Result	Expected Result	Sev / Prio
DEF-CAP-01	Register	Enter valid details with phone starting with '0'. <i>Actual:</i> Server 500 Error.	Account created successfully.	S2 / P1
DEF-CAP-02	Browse	Apply 'Asian' filter, then clear filter. <i>Actual:</i> Screen remains blank.	All restaurants displayed again.	S3 / P3
DEF-CAP-03	Order	Add \$25 item, apply SAVE5 coupon. <i>Actual:</i> \$10 deducted (applied twice).	Exactly \$5 discount deducted.	S1 / P1
DEF-CAP-04	Checkout	Select COD on \$120 cart. <i>Actual:</i> Order succeeds despite \$100 limit.	Error: "COD restricted > \$100".	S2 / P2
DEF-CAP-05	Tracking	Open live order tracking map. <i>Actual:</i> Driver icon missing (JS Error).	Driver pin icon displayed on map.	S3 / P2

5. Backend SQL Validation Queries

```
-- Query 1: Verify new user registration record creation in users table
-- Expected Result: 1 row returned matching user_id with correctly hashed password and
status='ACTIVE'.
SELECT user_id, email, phone_number, status, created_at
FROM users WHERE email = 'john.doe@example.com';

-- Query 2: Verify user delivery address association after registration
-- Expected Result: Stored pincode, city, and street match frontend input data.
SELECT address_id, user_id, street_address, city, pincode
FROM user_addresses WHERE user_id = 10293;

-- Query 3: Verify order record creation after order placement
-- Expected Result: Record exists with order_status = 'PLACED' and accurate total values.
SELECT order_id, user_id, restaurant_id, total_amount, order_status
FROM orders WHERE user_id = 10293 ORDER BY created_at DESC LIMIT 1;

-- Query 4: Verify order item mapping and quantities in DB
-- Expected Result: Purchased item IDs, quantities, and unit prices match cart checkout items.
SELECT order_item_id, order_id, item_id, quantity, unit_price
FROM order_items WHERE order_id = 50089;

-- Query 5: Verify payment transaction status and gateway response log
-- Expected Result: payment_status = 'COMPLETED' and transaction_id is non-null for online
payments.
SELECT payment_id, order_id, payment_method, payment_status, transaction_id
FROM payments WHERE order_id = 50089;
```

SECTION D — AI-AUGMENTED LEARNING

Step 1 · Build with AI

Exact Prompt Given to AI:

"Act as an expert Senior QA Automation & Manual Engineer. Write 10 test cases for a 'Schedule Order' feature of a food delivery application that allows users to place orders up to 7 days in advance. Follow standard format: Test Case ID, Description, Test Steps, Expected Result. Ensure coverage for valid future date/time scheduling, invalid inputs (past time, more than 7 days ahead, invalid date format), and order confirmation notification behavior."

Combined Test Suite (AI Original vs. Corrected & Extended Version)

TC ID	Description	Test Steps	Expected Result	Origin / Status
TC_SCH_01	Valid Future Date	Select date 3 days ahead at 14:00.	Order scheduled successfully.	AI Original
TC_SCH_02	Valid Same-Day Scheduling	Select time 2 hours ahead today.	Order scheduled successfully.	AI Original
TC_SCH_03	Past Date Selection	Select yesterday's date.	Error: "Cannot select past date".	AI Original
TC_SCH_04	Past Time Selection	Select time 30 mins prior to now.	Error: "Cannot select past time".	AI Original
TC_SCH_05	Day 8 Boundary Test	Select date 8 days from today.	Error: "Max 7 days allowed".	AI Original
TC_SCH_06	Confirmation Notification	Complete scheduled order booking.	Email/SMS confirmation sent.	AI Original
TC_SCH_07	Pre-Delivery Reminder	Advance clock to 30 mins before.	Push notification reminder sent.	AI Original
TC_SCH_08	Restaurant Closed Hours	Select slot when restaurant is closed.	Error: "Restaurant closed".	AI Original
TC_SCH_09	Invalid Date Format	Input date string 2026-13-45.	Error: "Invalid date format".	AI Original
TC_SCH_10	Cancellation Window	Cancel order 2 hours before slot.	Order canceled; refund sent.	AI Original
TC_SCH_11	Exact 7-Day Boundary	Select date exactly 7 days (168 hrs) ahead.	Order scheduled successfully.	ADDED BY QA
TC_SCH_12	Invalid Date (Feb 30)	Manually input/type 30th Feb 2026.	Rejection or auto-correction error.	ADDED BY QA
TC_SCH_13	Leap Year Boundary	Schedule for Feb 29 in non-leap year.	System blocks invalid leap date.	ADDED BY QA
TC_SCH_14	Min Lead Time Boundary	Select time 5 minutes ahead of now.	Error: "Min 30-min lead time required".	FIXED BY QA

Step 2 · Test & Debug (Without AI)

QA Critique & Review Note on AI Gaps vs. Human Judgment:

The AI tool provided a good foundational test suite but missed crucial domain boundary conditions: it tested Day 8 (invalid) but failed to test **Day 7 exact boundary (valid)** and ignored real-world business constraints like **minimum preparation lead time (30 mins)**. Furthermore, its calendar validation test cases were generic and omitted complex calendar edge cases such as **Feb 30** or leap-year date inputs.

Human judgment remains indispensable in test engineering because AI lacks contextual awareness of real-world operational domain rules (e.g., kitchen preparation lead times, timezone changes during 7-day windows, and exact boundary threshold mechanics).